

2-16

Date: 2/16/2026 at 12:00

Presenter: James Hurd

No presentation name

Summary

James Hurd is a former University of Kansas student and ACM leader (also a friend of mine). In his presentation, he shares his journey from graduating in Lawrence to becoming a software engineer at Microsoft in Seattle. He works for the Azure Core division, specifically focusing on host networking and software-defined accelerated technology. His presentation is about the gap between classroom concepts (like IP headers, Mac addresses, and virtualization) and their large-scale application in the global cloud infrastructure that supports massive clients like OpenAI, Walmart, and Lego.

Azure is Microsoft's cloud offering, providing compute hardware and virtualization through hypervisors like Hyper-V. James works on host networking, which encompasses the software and hardware on the server blade that interfaces with the physical network. Central to this architecture is "Azure Boost", which utilizes a custom network interface card to virtualize network resources. A key component of this system is the Virtual Filtering Platform, a software-based rules engine that handles policy processing, firewalls, and load balancing. To overcome the inherent latency of software processing, Microsoft employs Generic Flow Tables on FPGAs. This allows packet manipulation to occur at hardware speeds while remaining reprogrammable for bug fixes and performance updates.

Currently, James operates within the New Product Introduction (NPI) group. This team is dedicated to validating and verifying the functionality of new hardware before it is officially integrated into the Azure cloud. This involves testing high-end components such as Nvidia GPUs and CXL memory expansion cards to ensure they meet strict Service Level Agreements. The work is highly exploratory, requiring the team to define acceptable performance metrics and automate testing suites. This automation is critical because it reduces the time hardware spends in the validation phase, allowing it to reach customers faster.

On a daily basis, he utilizes a diverse toolkit including both Windows and Linux, PowerShell/Bash scripting, and Python for data analysis. He also uses low-level probing tools like eBPF and Wireshark to monitor kernel activity and packet captures. Despite working on a small team primarily composed of senior and principal engineers, he emphasizes that professional growth is driven by continuous curiosity. He describes the Microsoft hiring process as a rigorous path involving referrals, online assessments, and multiple rounds of technical interviews. James recaps his presentation by encouraging students to embrace the adventure

of moving to new environments and to never be afraid to ask questions when navigating complex, undocumented systems.

What I learnt

Industry roles frequently demand a mastery of tools and proprietary systems never encountered in a typical computer science curriculum. For example, managing global cloud traffic requires proficiency in niche query languages like Kusto and low-level kernel probing via eBPF. These skills are acquired through on-the-job immersion rather than textbooks. Professional networking at this scale involves managing the constant tension between thorough hardware validation and the aggressive business push to ship products to customers.

Additionally, I learnt about the concept of "on-call". "On-call" rotations introduces a high-stakes responsibility where engineers must troubleshoot live production workloads during off-hours, a practice rarely simulated in a classroom. Success in this environment often hinges on the ability to navigate high-seniority team dynamics, where a new graduate might be the only junior among industry veterans with decades of experience. Ultimately, the transition reveals that professional engineering is a hybrid of technical adaptability, stakeholder negotiation, and the use of physical organizational tools like whiteboards to manage an overwhelming, high-velocity workload.

What to do different

Instead of just following a professor's lead, I'm going to focus on how to learn concepts faster on my own. I won't limit myself to what's covered in a syllabus; I'll actively experiment with different ways to pick up new tech, like breaking things and fixing them in real-time. The goal is to stop waiting for a hand-holding explanation and start diving into documentation and senior devs' brains immediately. I'll treat learning as an aggressive trial-and-error process, making sure I can pivot from "zero knowledge" to "functional" in a matter of days. This is very similar to advice from a previous presenter (I believe Desai), where he said knowing how to learn will be the best trait in the future.